

M5 — Flight Software

Software de a bordo para CubeSat

Manual de la coleccion CubeSat (M5 / 11)

11 de mayo de 2026

Prefacio

El *flight software* (FSW) es el software que corre en el OBC del satellite y en cada subsistema con procesador. Este manual cubre como diseñarlo, estructurarlo, y operarlo.

Prerequisitos: M0 (CubeSat 101), Raspberry_Pi_5_Profundo.

Índice general

Prefacio	i
1. Arquitecturas de OBC	1
1.1. Microcontrolador vs SoC con Linux	1
2. Sistemas operativos	2
2.1. Para microcontroladores	2
2.2. Para SoCs	2
3. Frameworks de flight software	3
3.1. NASA cFS (Core Flight System)	3
3.2. NASA F'(FPrime)	3
3.3. KubOS	3
3.4. Custom (lo que estamos haciendo en INTISAT)	3
4. Tareas y concurrencia	4
4.1. Tareas en RTOS	4
4.2. Comunicacion inter-tarea	4
5. Telemetria y comandos	5
5.1. CCSDS Space Packet	5
5.2. Beacon	5
5.3. Housekeeping (HK)	5
5.4. Comandos uplink	5
6. FDIR — Fault Detection, Isolation, Recovery	7
6.1. Concepto	7
6.2. Las tres etapas	7
6.3. Niveles de severidad	7
7. Watchdog	8
7.1. Watchdog hardware	8
7.2. Watchdog software (systemd)	8
8. OTA — Over The Air Updates	9
8.1. Por que se necesita	9
8.2. Requisitos del OTA	9
8.3. Estrategias de OTA	9
9. Telemetria y comandos en INTISAT	10
9.1. Protocolo I2C	10
9.2. Chunking	10

10.Configuration management	11
10.1. Git en orbita	11
10.2. Tablas de parametros	11
11.Testing del flight software	12
11.1. Unit tests	12
11.2. Integration tests	12
11.3. HIL — Hardware In the Loop	12
11.4. Long duration test	12
Bibliografia	13

Capítulo 1

Arquitecturas de OBC

1.1. Microcontrolador vs SoC con Linux

Aspecto	Microcontrolador	SoC con Linux
Ejemplo	STM32, ESP32, ATmega	RPi 5, Jetson
Precio	5–20 USD	50–500 USD
Consumo	0.1–1 W	3–15 W
RAM	64–512 kB	1–16 GB
Tipo de software	firmware bare-metal o RTOS	sistema operativo
Determinismo	alto (controlable)	bajo (jitter del kernel)
Complejidad	baja	alta
Workforce	ingeniería embebida	devs Linux

Cuando usar cada uno:

- **Microcontrolador:** para subsistemas con tareas simples y deterministas. EPS, ADCS, COMMS suelen usar STM32.
- **SoC con Linux:** para payloads que necesitan procesamiento pesado (IA, vision, machine learning). Tu Pi 5 es un caso ejemplar.

Conexion con el INTISAT

En el INTISAT, la arquitectura típica sería:

- **OBC principal:** microcontrolador STM32 (determinístico).
- **Payload:** Raspberry Pi 5 con Linux (potencia de cálculo).
- **ADCS, COMMS, EPS:** cada uno con su propio microcontrolador.

Los buses I2C/CAN conectan todo.

Capítulo 2

Sistemas operativos

2.1. Para microcontroladores

Sin OS (bare-metal): solo loop principal + interrupciones. Util para subsistemas muy simples.

FreeRTOS: el RTOS mas popular en CubeSat. Tareas, queues, semaphores, timers. Open source.

Zephyr: alternativa moderna a FreeRTOS, soportado por Linux Foundation. Tiene drivers para muchos chips.

RTEMS: usado en mision aeroespacial real (Mars rovers, satellites NASA). Mas complejo de aprender.

2.2. Para SoCs

Linux: distribuciones tipicas en CubeSat:

- **Raspberry Pi OS (Bookworm):** el que usamos.
- **Yocto / OpenEmbedded:** para builds custom super optimizadas.
- **Buildroot:** similar a Yocto pero mas simple.
- **Ubuntu Server:** si necesitas paquetes mas modernos.

Capítulo 3

Frameworks de flight software

3.1. NASA cFS (Core Flight System)

El estandar de NASA. Arquitectura publish-subscribe con apps modulares. Usa publish/subscribe + tablas de configuracion + FDIR.

Pros: probado en misiones reales, ecosistema maduro.

Contras: complejo, curva de aprendizaje empinada.

URL: <https://github.com/nasa/cFS>

3.2. NASA F'(FPrime)

Framework moderno usado en MarCO e Ingenuity. Mas accesible que cFS. XML define los componentes, codigo auto-generado.

URL: <https://github.com/nasa/fprime>

3.3. KubOS

Framework dedicado a CubeSat. Descontinuado pero docs todavia utiles.

3.4. Custom (lo que estamos haciendo en INTISAT)

Python + systemd corriendo en Linux. Mas simple pero menos formal.

Cuidado

Para mision *academica*: custom esta bien. Para mision *comercial o critica*: usar framework formal (Fó cFS) para trazabilidad y certificacion.

Capítulo 4

Tareas y concurrencia

4.1. Tareas en RTOS

Una *tarea* es un hilo de ejecucion. Cada una tiene:

- **Prioridad:** las altas se ejecutan primero.
- **Stack:** memoria propia.
- **Estado:** running, ready, blocked, suspended.

Las tareas tipicas en un CubeSat:

Tarea	Prioridad	Periodo
Watchdog	alta	1 s
Telemetria HK	alta	1 s
Comms RX	alta	evento
Comms TX	media	evento
ADCS control	alta	100 ms
Payload	baja	event-driven

4.2. Comunicacion inter-tarea

Queues: cola FIFO de mensajes. Una tarea publica, otra consume. Es el patron mas usado.

Semaphores: sincronizacion entre tareas (binarios o contadores).

Mutexes: proteccion de recursos compartidos.

Notifications: senales directas tarea-tarea (mas rapido que queue).

Capítulo 5

Telemetria y comandos

5.1. CCSDS Space Packet

El estandar de empaquetado mas usado en aeroespacial:

Campo	Tamaño
Header primario	6 bytes
Header secundario	variable
Datos	variable
Trailer (CRC)	0-4 bytes

5.2. Beacon

Mensaje periodico que el satellite emite cada N segundos. Contiene estado basico (voltaje, temperatura, modo). Es el "latido" del satellite.

5.3. Housekeeping (HK)

Empaquetado de telemetria mas detallada que el beacon. Tipicamente:

- Voltajes de cada rail.
- Corrientes consumidas.
- Temperaturas de varios sensores.
- Estado de cada subsistema.
- Contadores de eventos.
- Codigos de error activos.

Periodo tipico: 1 cada 10 segundos durante operacion nominal.

5.4. Comandos uplink

Estructura tipica:

- Opcode (que hacer).
- Parametros.

- Checksum.
- Authentication (cuando aplica).

Conexion con el INTISAT

Tu I2C slave del INTISAT usa una variante simplificada de CCSDS: [CMD_BYTE] [LEN] [DATA...]. Esta bien para mision academica. Para una version mas profesional, podrias migrar a CCSDS completo.

Capítulo 6

FDIR — Fault Detection, Isolation, Recovery

6.1. Concepto

FDIR es el conjunto de mecanismos para detectar fallas, aislarlas, y recuperarse autónomamente. Sin FDIR un satélite *se muere* al primer glitch.

6.2. Las tres etapas

Detección: identificar que hay un problema. Se hace con:

- Limit checks (telemetría fuera de rango).
- Heartbeat checks (un subsistema no responde).
- Self-tests periódicos.
- Validación de checksums.

Aislamiento: identificar QUE subsistema falló (no propagar el error).

Recovery: aplicar acción correctiva:

- Re-intentar la operación.
- Reiniciar el subsistema.
- Cambiar a modo de respaldo.
- Pasar a SAFE_MODE.
- Esperar comando de tierra.

6.3. Niveles de severidad

- **INFO:** informativo, no requiere acción.
- **WARNING:** prestar atención pero no crítico.
- **ERROR:** requiere acción.
- **CRITICAL:** forzar SAFE_MODE.

Capítulo 7

Watchdog

7.1. Watchdog hardware

Es un timer externo al CPU. Si el CPU no le da "kick" cada N segundos, el watchdog resetea el sistema completo.

Por que importa: si el CPU se cuelga (loop infinito, deadlock, error de hardware), el watchdog es la unica forma de recuperarse sin intervencion.

En CubeSat: imprescindible. Sin watchdog hardware, el satellite se muere ante el primer cuelgue.

7.2. Watchdog software (systemd)

En Linux, systemd ofrece watchdog a nivel servicio:

```
[Service]
Type=notify
WatchdogSec=600
ExecStart=...
```

El servicio debe llamar `sd_notify("WATCHDOG=1")` regularmente. Si no lo hace en 600 s, systemd lo reinicia.

Conexion con el INTISAT

Tu `cubesat-pipeline.service` ya usa `WatchdogSec=600` (ver `cubesat/systemd/*.service`). Eso protege contra cuelgues del pipeline IA.

Capítulo 8

OTA — Over The Air Updates

8.1. Por que se necesita

Despues del lanzamiento no se puede mas modificar el hardware. El unico modo de mejorar el satelite es **actualizar el software**.

8.2. Requisitos del OTA

- **Atomico**: una actualizacion exitosa, o sino el sistema queda en estado previo.
- **Verificable**: validar el binario antes de aplicarlo.
- **Recuperable**: si falla, rollback automatico al estado previo.
- **Resistente a corte**: si se interrumpe (perdida de pase, muerte de bateria), se reintenta.

8.3. Estrategias de OTA

A/B partitioning: dos particiones identicas. Al actualizar, se escribe en la inactiva. Si todo OK, se cambia el bootloader para usar la nueva. Si falla, vuelve a la vieja.

Git fetch + checkout: lo que usas en el INTISAT. Simple pero requiere disco con historico.

Delta updates: solo transmitir las diferencias. Ahorra bandwidth.

Conexion con el INTISAT

Tu `cubesat/ota.py` implementa git fetch + checkout con health check + rollback. Esta validado.

Capítulo 9

Telemetria y comandos en INTISAT

9.1. Protocolo I2C

El I2C slave en 0x42 responde a 14 comandos:

- `CMD_GET_STATUS` (0x01): estado actual.
- `CMD_START_CAPTURE` (0x02): dispara scan.
- `CMD_GET_THUMBNAIL` (0x06): bajada chunked de imagen.
- `CMD_GET_TELEMETRY` (0x05): bajada de telemetria.
- `CMD_SAFE_MODE` (0x10): pasar a safe.
- `CMD_RESUME` (0x11): salir de safe.
- `CMD_OTA_PREPARE` (0x20): preparar update.
- `CMD_OTA_COMMIT` (0x21): aplicar update.
- `CMD_REBOOT` (0x30): reboot suave.

9.2. Chunking

El I2C limita transferencias a 32 bytes. Para data mayor (thumbnail 100 KB), se trocea en frames de 30 bytes con flag `more=1/0`.

Capítulo 10

Configuration management

10.1. Git en orbita

Tu daemon usa git para OTA. Eso te da:

- Historial completo de cambios.
- Capacidad de rollback a cualquier commit.
- Branches para testing.
- Tags para versiones de vuelo.

10.2. Tablas de parametros

Como en cFS, se pueden cargar tablas de configuracion en runtime:

- Limites para FDIR.
- Coeficientes de calibracion.
- Schedule de tareas.
- Tunable parameters del pipeline.

Esto permite ajustar comportamiento sin actualizar codigo.

Capítulo 11

Testing del flight software

11.1. Unit tests

Probar cada modulo aislado. Pytest, GoogleTest, etc.

11.2. Integration tests

Probar la interaccion entre modulos.

11.3. HIL — Hardware In the Loop

Conectar el flight software a una simulacion del hardware (sensores y actuadores virtuales).
Permite probar logica completa sin satellite real.

11.4. Long duration test

Burn-in: correr el FSW durante 168 horas (1 semana) sin reboot. Verifica que no hay memory leaks, deadlocks acumulativos, etc.

Bibliografia

1. NASA. *cFS documentation*. <https://github.com/nasa/cFS>.
2. NASA. *F'Tutorials*. <https://nasa.github.io/fprime/>.
3. CCSDS. *Space Packet Protocol Recommendation*. <https://public.ccsds.org>.
4. FreeRTOS. *Official documentation*. <https://www.freertos.org>.
5. NASA Software Engineering Handbook. <https://swehb.nasa.gov>.